

Conversion Excel vers Supabase

Strategie complete de migration des 35 fichiers Excel (31 domaines, 620 tables) vers PostgreSQL 17.6 sur Supabase.
Detection et correction des erreurs de calcul, mapping des types, et garantie d'integrite des donnees RH.

Projet ADMINA-RH
Document technique de reference
Juillet 2026

1. Introduction et Contexte du Projet	2
2. Analyse des Fichiers Excel Sources	2
2.1 Structure Generale des Fichiers	2
2.2 Typologie des Formules Identifiees	3
3. Processus de Conversion Excel vers Supabase	4
3.1 Phase 1 : Extraction et Inventaire	4
3.2 Phase 2 : Mapping Feuille vers Table SQL	4
3.3 Phase 3 : Generation des Migrations SQL	5
4. Detection et Gestion des Erreurs de Calcul	5
4.1 Types d'Erreurs Identifiables	5
4.2 Methode de Detection en Quatre Couches	6
4.3 Traitement Specifique des Erreurs Courantes	7
5. Strategie de Migration des Donnees	7
5.1 Ordre de Migration	7
5.2 Gestion du Multi-Tenancy	8
5.3 Transactions et Rollback	8
6. Validation et Garantie de Qualite	9
6.1 Tests d'Integrite Post-Migration	9
6.2 Audit Continue et Monitoring	9
7. Synthese de la Methodologie	9

1. Introduction et Contexte du Projet

Le projet ADMINA-RH constitue une refonte complete du systeme de gestion des ressources humaines, structure autour de 31 domaines fonctionnels (D1 a D31) couvrant l'ensemble du cycle de vie employe, depuis le recrutement jusqu'a la retraite. Chaque domaine a ete prealablement modelise sous forme de fichiers Excel detailles, comprenant des feuilles de calcul, des tableaux de bord dynamiques, des formules de calcul complexes et des listes de reference. Ces fichiers representent le socle fonctionnel a partir duquel la base de donnees Supabase sera construite.

L'objectif de ce document est d'expliquer en detail la methodologie qui sera employee pour convertir ces fichiers Excel en structures de base de donnees relationnelles sur Supabase (PostgreSQL 17.6). Il aborde egalement les mecanismes de detection et de correction des erreurs de calcul ou de formule pouvant exister dans les feuilles Excel, afin de garantir l'integrite des donnees migrees. Cette demarche est essentielle car une base de donnees construite a partir de donnees erronees propagerait ces erreurs dans tout le systeme, avec des consequences potentiellement gravissimes sur la paie, les contrats, les conges et l'ensemble des processus RH.

L'inventaire actuel compte plus de 35 fichiers Excel couvrant les 31 domaines, avec environ 620 tables ou feuilles de calcul identifiees. Chaque fichier contient entre 5 et 21 feuilles, incluant des donnees de reference, des formules de calcul (COUNTIF, VLOOKUP, IFERROR, SUM, etc.), des validations de donnees et des mises en forme conditionnelles. La complexite de cette conversion requiert une approche systematique et rigoureuse, que nous allons detailler dans les sections suivantes.

2. Analyse des Fichiers Excel Sources

2.1 Structure Generale des Fichiers

Chaque fichier Excel de domaine suit une structure standardisee qui facilite la conversion automatique. Un domaine typique comme le Domaine 2 (Gestion Administrative du Personnel) contient 21 feuilles organisees de maniere logique : un tableau de bord synthetique, des feuilles de donnees principales (fiches employes, contrats, avenants), des feuilles de suivi (documents, conges, absences, heures supplementaires) et une feuille de listes de reference (_Lists). Cette organisation hierarchique se revele directement transposable en un schema relationnel ou chaque feuille principale devient une table et les listes de reference deviennent des tables d'enumeration.

Les fichiers contiennent trois types de donnees distincts qu'il faut traiter differemment lors de la conversion. Premierement, les donnees statiques (noms, dates, montants, identifiants) qui se transposent directement en colonnes de tables SQL. Deuxiemement, les formules de calcul (duree de contrat, anciennete, solde de conges, montants de cotisations) qui doivent etre reimplementees sous forme de colonnes generees (generated columns) ou de vues SQL. Troisiemement, les regles de validation et mises en forme conditionnelles qui se traduisent en contraintes CHECK, triggers ou regles RLS dans Supabase.

Domaine	Feuilles	Exemples de Formules	Complexite
D2 - Admin. Personnel	21	COUNTIF, IFERROR, calcul anciennete	Moyenne
D3 - Paie/Remuneration	21	VLOOKUP, SUM, cotisations sociales	Tres elevee
D5 - Conges/Absences	~15	COUNTIF, calcul solde, date limite	Elevee
D7 - Performance	~12	IFERROR, moyennes ponderes	Moyenne
D1 - Recrutement	~15	COUNTIF, calcul delais	Moyenne

2.2 Typologie des Formules Identifiees

L'analyse approfondie des fichiers a revele plusieurs categories de formules Excel, chacune necessitant une strategie de conversion specifique. La comprehension de ces categories est fondamentale pour garantir que la logique metier encodee dans les formules Excel soit fidelement reproduite dans la base de donnees PostgreSQL.

Formules d'agregation (COUNTIF, SUMIF, AVERAGEIF) : Ces formules comptent ou somment des lignes en fonction de criteres. Par exemple, le tableau de bord du D2 utilise COUNTIF pour compter le nombre d'employes actifs ou de cadres. En SQL, ces formules se traduisent naturellement par des requetes d'agregation (SELECT COUNT(*), SUM(), AVG()) avec des clauses WHERE pour les criteres. Elles seront implementees sous forme de vues materielisees ou de fonctions PL/pgSQL pour le tableau de bord.

Formules de recherche (VLOOKUP, INDEX/MATCH) : Utilisees pour recuperer des donnees d'une feuille a une autre, par exemple pour lier un matricule employe a sa categorie salariale ou a son departement. En SQL, ces formules se traduisent par des jointures (JOIN) entre tables, ou par des sous-requetes correes dans les cas les plus complexes. L'avantage de la base de donnees est que ces relations deviennent structurelles (cles etrangeres) plutot que positionnelles.

Formules conditionnelles (IF, IFERROR, AND, OR) : Elles implementent la logique metier comme la verification de dates de fin de contrat, le calcul d'anciennete avec gestion des erreurs, ou la determination du statut d'un document. En SQL, ces formules deviennent des expressions CASE, des fonctions COALESCE/NULLIF, ou des contraintes CHECK. Par exemple, la formule =IFERROR((TODAY()-S5)/365.25, 0) qui calcule l'anciennete sera convertie en une colonne generee avec la fonction EXTRACT(YEAR FROM AGE(date_embauche)).

Formules de calcul arithmetique : Multiplications, divisions, soustractions pour les montants de paie, les cotisations, les indemnites. La conversion directe en SQL est generalement straightforward, mais la precision numerique doit etre soigneusement geree : Excel utilise un formatage decimal qui peut masquer des erreurs d'arrondi, tandis que PostgreSQL offre les types NUMERIC et DECIMAL pour une precision exacte. Nous utiliserons systematiquement NUMERIC(p,s) pour tous les montants financiers.

3. Processus de Conversion Excel vers Supabase

3.1 Phase 1 : Extraction et Inventaire

La premiere etape consiste a extraire de maniere programmatique l'ensemble des metadonnees de chaque fichier Excel. Nous utilisons la bibliotheque Python openpyxl en mode double : une premiere passe avec data_only=True pour lire les valeurs calculees, et une seconde passe sans ce flag pour capturer les formules brutes. Cette double lecture permet de disposer a la fois du resultat numerique et de la logique de calcul, ce qui est indispensable pour la validation croisee.

Pour chaque feuille, nous extrayons : le nom de la feuille, la plage de donnees (dimensions), les en-tetes de colonnes (ligne 4 ou 5 generalement), le type de donnee de chaque colonne (texte, date, numerique, booleen), le nombre de lignes de donnees, la liste des formules presentes avec leurs coordonnees, et les regles de validation associees. Ces metadonnees sont stockees dans un registre d'inventaire au format JSON qui servira de reference tout au long du processus de migration.

L'analyse automatique detecte egalement les dependances inter-feuilles : quand une formule dans la feuille A reference des cellules de la feuille B, cette dependance est enregistree pour garantir que les tables correspondantes seront creees dans le bon ordre lors de la generation des migrations SQL.

3.2 Phase 2 : Mapping Feuille vers Table SQL

Chaque feuille de donnees est mappee vers une table SQL selon des regles de conversion systematiques. Le nom de la feuille est transforme en nom de table en respectant les conventions PostgreSQL (minuscules, underscores, pas d'espaces ni d'accents). Par exemple, la feuille '2-Fiche Employe' devient la table 'd2_fiche_employe'. Chaque colonne Excel est analysee pour determiner son type SQL le plus approprie, en tenant compte des donnees reelles presentes et non seulement du nom de l'en-tete.

Type Excel	Type SQL	Exemple	Notes
Texte (string)	VARCHAR(n) ou TEXT	Nom, Prenom, Adresse	VARCHAR(255) par default
Date (datetime)	DATE ou TIMESTAMP	Date Naissance, Date Debut	DATE pour dates seules
Numerique entier	INTEGER ou BIGINT	Matricule, Nombre enfants	BIGINT pour IDs
Numerique decimal	NUMERIC(p,s)	Salaire, Taux cotisation	p=15, s=2 pour montants
Booleen	BOOLEAN	Actif, Cadre	Oui/Non vers true/false
Pourcentage	NUMERIC(5,2)	Taux horaire, Prime %	Stocke en valeur decimale
Monetaire (FCFA)	NUMERIC(15,2)	Montant paie, Indemnite	Pas de centimes en FCFA
Formule (calcul)	Col. generee ou Vue	Anciennete, Duree contrat	STORED ou IMMUTABLE

Pour les colonnes contenant des listes de valeurs predefinies (comme le type de contrat : CDI, CDD, Interim, Stage), nous creons des tables d'enumeration separees avec une contrainte de cle etrangere. Cette approche respecte la troisieme forme normale (3NF) et facilite les mises a jour futures sans avoir a modifier les donnees existantes. La feuille _Lists presente dans chaque fichier Excel sert de source pour ces tables de reference.

3.3 Phase 3 : Generation des Migrations SQL

Une fois le mapping etabli, un script Python genere automatiquement les fichiers de migration SQL au format Supabase. Chaque migration suit le schema : CREATE TABLE avec toutes les colonnes typeees, les contraintes PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, les indexes recommandes, les triggers pour les colonnes calculees, et les politiques RLS (Row Level Security) pour le multi-tenancy via tenant_id. Les migrations sont generees dans un ordre topologique respectant les dependances entre tables.

Les noms de colonnes sont normalises : les accents sont supprimes, les espaces remplaces par des underscores, tout est converti en minuscules. Par exemple, 'N° Matricule' devient 'numero_matricule', 'Date Naissance' devient 'date_naissance', 'Salaire Brut' devient 'salaire_brut'. Ce processus de normalisation garantit la coherence et la maintenabilite du schema.

Pour les colonnes calculees, nous utilisons les colonnes generees de PostgreSQL (GENERATED ALWAYS AS). Par exemple, la colonne 'anciennete_annees' sera definie comme GENERATED ALWAYS AS (EXTRACT(YEAR FROM AGE(date_embauche))) STORED. Le mot-cle STORED est prefere a IMMUTABLE car il permet de creer des indexes sur ces colonnes, ce qui est essentiel pour les requetes de recherche et de tri sur l'anciennete des employes.

4. Detection et Gestion des Erreurs de Calcul

4.1 Types d'Erreurs Identifiables

Les fichiers Excel, malgre leur utilite pour la modelisation, sont inheremment sujets a plusieurs categories d'erreurs qui peuvent se propager silencieusement. La detection de ces erreurs est un enjeu critique car la base de donnees finale doit etre exempte de toute incoherence. Nous avons identifie six categories principales d'erreurs a traiter systematiquement avant et pendant la conversion.

Type d'Erreur	Exemple Concret	Impact si Non Detectee	Methode de Detection
Reference circulaire	Cellule A1 = B1 + C1; B1 = A1 * 2	Valeurs incorrectes, calculs infinis	openpyxl: data_only=True retourne 0
Division par zero	=Montant/Quantite avec Quantite=0	Erreur #DIV/0! propagee en cascade	IFERROR dans la formule (bon signe)

Type d'Erreur	Exemple Concret	Impact si Non Detectee	Methode de Detection
Incoherence de type	Texte dans une colonne numerique	Rejet SQL a l'insertion	Validation de type par colonne
Formule corrompue	Reference a une cellule supprimee #REF!	Donnees manquantes silencieuses	Scan des valeurs #REF!, #N/A, #VALUE!
Erreur d'arrondi	Somme de pourcentages > 100%	Ecart dans les masses salariales	Comparaison somme vs total attendu
Dependance brisee	VLOOKUP vers une feuille inexistante	Donnees incorrectes non visibles	Analyse des references inter-feuilles

4.2 Methode de Detection en Quatre Couches

Notre approche de detection des erreurs repose sur quatre couches complementaires qui s'executent sequentiellement. Chaque couche attrape une categorie specifique d'erreurs, et leur combinaison garantit une couverture quasi-totale des cas de figure possibles. Cette architecture en couches est inspiree des principes de defense en profondeur utilises en securite informatique.

Couche 1 - Audit Statique des Formules : Un script Python parcourt chaque cellule formulee et verifie les proprietes suivantes : presence d'IFERROR ou ISERROR (bonne pratique), references vers des plages valides (pas de #REF!), types de donnees coherents avec le resultat attendu, et absence de divisions par zero potentielles. Pour chaque formule detectee, le script extrait les references de cellules et les verifie contre l'inventaire des feuilles. Si une reference pointe vers une feuille qui n'existe pas dans le fichier, une alerte est generee.

Couche 2 - Validation Croisee Valeurs Calculees vs Formules : Pour chaque cellule contenant une formule, nous comparons la valeur calculee par Excel (obtenue via openpyxl en mode data_only) avec le resultat de la reexecution de la formule en Python. Cette reexecution est possible pour les formules simples (operations arithmetiques, IF, COUNTIF) mais pas pour les formules complexes impliquant des fonctions specifiques a Excel. En cas d'ecart superieur a un seuil de tolerance (0.01 pour les montants, 1 jour pour les dates), une alerte est emise avec le detail de la formule, la valeur Excel et la valeur recalculee.

Couche 3 - Contraintes d'Integrite dans PostgreSQL : Apres la migration des donnees, des contraintes CHECK sont ajoutees sur toutes les colonnes sensibles pour attraper les donnees anormales qui auraient echappe aux deux premieres couches. Par exemple : CHECK (salaire_brut > 0), CHECK (date_fin IS NULL OR date_fin > date_debut), CHECK (pourcentage BETWEEN 0 AND 100), CHECK (anciennete_annees >= 0). Ces contraintes agissent comme un filet de securite ultime qui rejette toute donnee incoherente au moment de l'insertion.

Couche 4 - Tests de Regression Automatisees : Des scripts de test comparent les agregations globales entre les fichiers Excel source et les tables SQL cible. Par exemple, le nombre total d'employes dans la

table d2_fiche_employe doit correspondre au COUNTIF du tableau de bord Excel. La somme des salaires bruts dans la base doit etre egale au SUM de la colonne correspondante dans Excel (a un seuil d'arrondi pres). Ces tests sont integres dans le pipeline CI/CD et s'executent a chaque modification du schema ou des donnees.

4.3 Traitement Specifique des Erreurs Courantes

Pour chaque type d'erreur identifiee, nous avons defini un protocole de traitement specifique. L'objectif n'est pas simplement de supprimer les erreurs, mais de comprendre leur cause racine et de s'assurer que la logique metier est correctement traduite dans la base de donnees. Voici les traitements applicables aux erreurs les plus frequentes.

Erreurs #REF! (references brisees) : Lorsqu'une formule reference une cellule qui a ete supprimee ou deplacee, Excel affiche #REF!. Dans notre pipeline, ces erreurs sont detectees par un scan systematique de toutes les valeurs de cellules. Si une erreur #REF! est trouvee, le script identifie la formule originale, tente de reconstituer la reference correcte en analysant le contexte (formules similaires dans les lignes adjacentes), et genere un rapport d'alerte. La donnee est marquee comme 'a verifier manuellement' dans le registre de migration.

Valeurs #N/A (donnees introuvables) : Les recherches VLOOKUP qui echouent retournent #N/A. Cela peut indiquer soit une donnee manquante legitime (un employe sans affectation de departement), soit une erreur de saisie (un identifiant mal orthographie). Notre script distingue ces deux cas en verifiant si la cle de recherche existe dans la table cible. Si la cle n'existe pas, c'est une erreur de reference qu'il faut corriger. Si la cle existe mais retourne vide, c'est une donnee manquante legitime que nous traduisons par NULL en SQL.

Erreurs d'arrondi cumulatif : Les calculs de paie impliquent de nombreuses operations arithmetiques successives qui peuvent accumuler des erreurs d'arrondi. Par exemple, si un salaire horaire de 5 416,67 FCFA est multiplie par 173,33 heures, le resultat exact est 939 445,33 FCFA, mais Excel peut afficher 939 445,32 FCFA a cause de la representation interne en virgule flottante. Pour prevenir ce probleme, nous utilisons le type NUMERIC(15,2) dans PostgreSQL qui effectue tous les calculs en precision decimale exacte, et nous definissons des fonctions PL/pgSQL pour les calculs de paie qui arrondissent systematiquement a 2 decimales apres chaque operation.

5. Strategie de Migration des Donnees

5.1 Ordre de Migration

La migration suit un ordre topologique strict base sur les dependances entre tables. Les tables de reference (listes de valeurs, categories, departements) sont migrees en premier, suivies des tables principales (employes, contrats) qui dependent des references, puis des tables de transaction (conges, absences, paie) qui dependent des tables principales. Enfin, les vues et tableaux de bord sont crees apres toutes les tables

de donnees. Cet ordre garantit que chaque contrainte de cle etrangere peut etre satisfaite au moment de l'insertion.

Ordre	Type de Table	Exemples	Volume Estime
1	Tables de reference	types_contrat, statuts_employe, categories	10-50 lignes/table
2	Tables principales	employes, contrats, postes	100-10 000 lignes
3	Tables de transaction	conges, absences, bulletins_paie	1 000-100 000 lignes
4	Tables de liaison	competences_employe, documents Employe	Variable
5	Vues et fonctions	v_tableau_bord_d2, f_calcul_paie	Generees automatiquement

5.2 Gestion du Multi-Tenancy

L'application ADMINA-RH etant multi-tenant, chaque table inclut obligatoirement une colonne `tenant_id` (UUID) qui identifie l'organisation a laquelle appartiennent les donnees. Les politiques RLS (Row Level Security) de Supabase sont configurees pour filtrer automatiquement toutes les requetes en fonction du `tenant_id` de l'utilisateur connecte. Cela signifie que les donnees migrees depuis les fichiers Excel seront associees a un `tenant_id` par defaut lors de l'import initial, puis reattribuees aux tenants appropries lors de la configuration de chaque organisation.

Les triggers PostgreSQL sont utilises pour garantir que le `tenant_id` est toujours renseigne a l'insertion (valeur par defaut derivee du contexte JWT de la session) et qu'il ne peut pas etre modifie apres creation (trigger BEFORE UPDATE qui empeche la modification du `tenant_id`). Cette protection au niveau de la base de donnees constitue une couche de securite supplementaire par rapport aux seuls controles applicatifs.

5.3 Transactions et Rollback

Chaque domaine est migre dans une transaction SQL atomique. Si une erreur survient pendant l'insertion des donnees d'un domaine (violation de contrainte, type incoherent, donnee manquante obligatoire), la transaction entiere est annulee (ROLLBACK) et un rapport detaille est genere identifiant la ligne et la colonne problematique. Cette approche garantit que la base de donnees reste dans un etat coherent a tout moment, sans donnees partielles qui pourraient corrompre les requetes et les calculs par la suite.

Pour les tres grands volumes de donnees (plus de 10 000 lignes), nous utilisons la commande COPY de PostgreSQL qui est nettement plus performante que des INSERT individuels. Avant chaque import COPY, un fichier temporaire CSV est genere a partir des donnees Excel, puis valide contre le schema de la table cible. Les lignes rejetees sont enregistrees dans un fichier de rejet separe avec le motif de rejet, ce qui permet un traitement manuel ulterieur.

6. Validation et Garantie de Qualite

6.1 Tests d'Integrite Post-Migration

Apres chaque migration de domaine, une batterie de tests automatiques est executee pour verifier l'integrite des donnees transferees. Ces tests comparent les indicateurs cles entre la source Excel et la cible SQL, en s'appuyant sur les tableaux de bord presents dans chaque fichier Excel comme reference de verite. Tout ecart superieur au seuil de tolerance declenche une investigation automatique et un rapport d'anomalie.

Test	Source Excel	Cible SQL	Tolerance
Comptage total	COUNTIF sur plage	SELECT COUNT(*)	0 (exact)
Somme montants	SUM sur colonne	SELECT SUM(col)	+/-1 FCFA
Moyennes	AVERAGE sur plage	SELECT AVG(col)	+/-0.01
Comptage conditionnel	COUNTIF avec critere	SELECT COUNT(*) WHERE	0 (exact)
Unicite des cles	Pas de doublons visuels	UNIQUE constraint	0 doublon
Referentiels	Listes de validation	FOREIGN KEY check	0 orphelin

6.2 Audit Continue et Monitoring

Une fois la base de donnees en production, un systeme d'audit continu est mis en place pour detecter toute anomalie qui pourrait survenir apres la migration initiale. Ce systeme comprend des triggers de verification sur les tables sensibles (paie, contrats) qui comparent chaque nouvelle valeur inseree ou modifiee avec les regles metier definies, des fonctions PL/pgSQL d'audit executees periodiquement (quotidiennement pour la paie, hebdomadairement pour les conges) qui verifient la coherence globale des donnees, et des alertes automatiques envoyees a l'administrateur RH lorsqu'une incoherence est detectee.

Supabase offre des fonctionnalites natives qui facilitent ce monitoring : les Realtime subscriptions permettent de surveiller les changements de donnees en temps reel, les Database Webhooks peuvent declencher des notifications externes, et les logs de requetes permettent de tracer l'origine de chaque modification. En combinant ces outils avec nos propres fonctions d'audit, nous disposons d'un systeme de surveillance complet qui garantit la fiabilite des donnees RH a long terme.

7. Synthese de la Methodologie

La conversion des fichiers Excel vers la base de donnees Supabase suit un pipeline rigoureux en sept etapes, chacune incluant ses propres mecanismes de verification et de validation. Ce pipeline est concu pour etre reproductible, automatisable et auditable, garantissant que chaque donnee migree est fidele a sa

source Excel et conforme aux regles metier du systeme ADMINA-RH.

Etape	Description	Outil	Produit
1. Inventaire	Extraction des metadonnees de chaque fichier	Python + openpyxl	Registre JSON
2. Audit Formules	Detection des erreurs et incoherences	Script Python	Rapport d'anomalies
3. Mapping	Association feuille Excel vers table SQL	Script Python	Schema de mapping
4. Normalisation	Nettoyage des noms et types de donnees	Script Python	Donnees nettoyees
5. Generation SQL	Creation des migrations Supabase	Python + templates	Fichiers .sql
6. Migration	Insertion des donnees dans Supabase	Supabase CLI + psql	Tables peuplees
7. Validation	Tests d'integrite post-migration	Scripts de test	Rapport de conformite

Les quatre couches de detection des erreurs (audit statique, validation croisee, contraintes SQL, tests de regression) garantissent que les erreurs de calcul et de formule presentes dans les fichiers Excel sont detectees et corrigees avant leur integration dans la base de donnees. L'utilisation de colonnes generees, de vues SQL et de fonctions PL/pgSQL permet de reproduire fidelement la logique metier des formules Excel tout en beneficiant de la robustesse, la performance et la coherence transactionnelle d'une base de donnees relationnelle professionnelle. Cette approche methodique assure une transition fiable depuis l'environnement Excel de conception vers la plateforme operationnelle Supabase.